```
void drawButtons() {
  int buttonRadius = 30;

  // Draw Call Button
  int callButtonX = 80;
  int callButtonY = 180; // Moved up
  tft.fillCircle(callButtonX, callButtonY, buttonRadius, TFT_GREEN);
  tft.setTextColor(TFT_WHITE, TFT_GREEN);
  tft.setCursor(callButtonX - 15, callButtonY - 7); // Center text inside the circle
  tft.print("Call");

  // Draw End Button
  int endButtonX = 160;
  int endButtonY = 180; // Moved up
  tft.fillCircle(endButtonX, endButtonY, buttonRadius, TFT_RED);
  tft.setTextColor(TFT_WHITE, TFT_RED);
  tft.setCursor(endButtonX - 15, endButtonY - 7); // Center text inside the circle
  tft.print("End");
}

void loop() {
  if (touch.available()) {
    int x = touch.data.x;
    int y = touch.data.y;
    if (isCallButtonTouched(x, y)) {
      // Handle Call button press
      Serial.println("Call button pressed");
    }
    if (isEndButtonTouched(x, y)) {
      // Handle End button press
      Serial.println("End button pressed");
```

Fig. 3: Code snippet for calling screen UI design

```
      row++;
    }
  }
}

void loop() {
  if (touch.available()) {
    int x = touch.data.x;
    int y = touch.data.y;
    text += getKey(x, y);
    tft.setCursor(30, 80);
    tft.print(text + "_");
  }
}

String getKey(int x, int y) {
  int keyWidth = 28;
  int keyHeight = 28;
  int startX = 60; // Adjusted startX
  int startY = 120;

  int col = (x - startX) / (keyWidth + 5);
  int row = (y - startY) / (keyHeight + 5);

  int index = row * 3 + col;
  if (index >= 0 && index < 12) {
    return keys[index];
  } else {
    return "";
  }
}
```

Fig. 5: Code snippet for dial pad UI design for the smallest phone

Fig. 4: Dial pad UI

sharing, Wi-Fi-based calling, and a phone finder were added after completing the design. The smartphone can be programmed with more functions, if needed. This first part of the multi-part article focuses on designing the basic UI and testing the GSM module.

EFY note. Before proceeding with the UI code, install ESP32 on Arduino IDE. Then, install the libraries for TFT_eSPI display, GSMSim for interfacing the SIM card module, and CTS816 for using touch on the display.

Designing UI

The basic UI will cover functions such as displaying numbers, attending calls, typing messages, and sending messages. The calling UI for the incoming call screen will be designed first. The incoming call number will be displayed, and two UI touch buttons will be added for attending or cancelling the call. The UI design will include code to display the number and two buttons with functions to detect touchpoints on the buttons displayed on the screen. Fig. 2 shows the incoming call screen UI design. Fig. 3 shows the code snippet for the calling screen UI design.

Dial pad UI

The next step is designing the dial screen UI. Various touch buttons for different numbers will be added.

Fig. 6: UI for message and keypad

The code will check the touchpoints for each number and determine whether the touchpoint matches the range of the buttons displayed on the screen. A function will store the number as a string and display it as the dialled number. Fig. 4 shows the dial pad UI. Fig. 5 shows the code snippet for the dial pad UI design for the smallest phone.